

Visual Computing Project Report

Okan Güclü

February 6, 2026

Introduction

Traditional real-time rendering pipelines based on rasterization are inherently limited and cannot accurately reproduce global light transport effects such as indirect illumination, soft shadows, or multiple light bounces. While physically based rendering techniques such as path tracing can simulate these effects in a physically correct manner, they are typically associated with high computational cost and often rely on modern GPUs featuring dedicated ray tracing hardware.

The primary motivation of this project was to implement a physically based path tracer that runs on GPUs without hardware ray tracing support. Since standard OpenGL does not provide native ray tracing functionality, the goal was to design a fully shader-based solution using compute shaders. The renderer evaluates the rendering equation stochastically and progressively converges to a physically plausible image by accumulating samples over multiple frames.

The developed renderer implements a physically based shading pipeline with energy-conserving materials [5], **microfacet BRDFs** [7, 2], **MIS (Multiple Importance Sampling)** [5, 6], and **BVH-accelerated ray traversal** [5]. The system converges to stable solutions over time and generates visually appealing global illumination effects without the need for hardware ray tracing units. I was the only student that worked on the project; I was in charge of the renderer design, shader implementation, and experimental evaluation.

The complete implementation, including shaders and example scenes ¹, is publicly available as open-source code.²

Background

Real-time shading models Traditional real-time shading models, such the Blinn-Phong lighting model, combine diffuse and specular reflection terms to approximate surface appearance. These models are computationally efficient, but they lack realism in complicated environments since they only take into account direct illumination and overlook light interactions between surfaces.

Physically based rendering Physically based rendering aims to model light transport according to physical principles [5]. Surface reflection is described by a bidirectional reflectance distribution function (BRDF), which determines how incoming light is scattered into outgoing directions depending on material properties and surface orientation [5].

Rendering equation The interaction of light with surfaces is formally described by the rendering equation [5]:

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i \quad (1)$$

where L_o is the outgoing radiance, L_e is emitted radiance, f_r is the BRDF, ω_i and ω_o are incoming and outgoing directions, and $\mathbf{n}$ denotes the surface normal.

Monte Carlo integration The rendering equation (Eq. 1) involves an integral over all incoming light directions, which cannot be evaluated analytically for general scenes. Path tracing therefore approximates this integral using Monte Carlo integration by randomly sampling directions on the hemisphere above the surface [5].

Using N random samples ω_i^k drawn from a probability density function $p(\omega_i)$, the outgoing radiance can be estimated as:

$$L_o(\mathbf{x}, \omega_o) \approx \frac{1}{N} \sum_{k=1}^N \frac{f_r(\mathbf{x}, \omega_i^k, \omega_o) L_i(\mathbf{x}, \omega_i^k) (\omega_i^k \cdot \mathbf{n})}{p(\omega_i^k)} \quad (2)$$

¹<https://casual-effects.com/data/index.html>

²<https://github.com/AAstroPhysiCS/PathTracer>

Here, ω_i^k denotes the k -th sampled incoming direction, and $p(\omega_i^k)$ is the probability density with which this direction was generated. The term $f_r(\mathbf{x}, \omega_i^k, \omega_o)$ is the BRDF. The factor $(\omega_i^k \cdot \mathbf{n})$ accounts for the cosine falloff, and $L_i(\mathbf{x}, \omega_i^k)$ represents the incident radiance arriving from direction ω_i^k . [5]

The estimator is unbiased if the expected value of the estimate converges to the actual solution of the integral, which is ensured by the division by $p(\omega_i^k)$. [5].

In practice, the choice of the sampling distribution $p(\omega_i)$ strongly affects convergence speed. Importance sampling selects directions according to distributions that resemble the shape of the integrand, thereby reducing variance. This principle forms the basis for cosine-weighted hemisphere sampling for diffuse reflection, VNDF sampling for GGX specular reflection, and their combination using multiple importance sampling [5, 6].

Bounding Volume Hierarchy In scenes with hundreds of thousands to millions of triangles, path tracing speed depends on efficient ray-scene intersection. By grouping primitives into a tree of nested axis-aligned bounding boxes (AABBs), a bounding volume hierarchy (BVH) speeds up ray tracing. While leaf nodes hold a limited collection of primitives, each internal node encloses the bounding boxes of its offspring. Rays are first checked against bounding volumes during traversal; child nodes or primitives are only assessed if an intersection takes place. When compared to linear traversal, this lowers the typical amount of costly triangle tests. [5]

Along each axis, the ray is intersected with slabs defined by the box planes to calculate the ray-AABB intersection [5]. This project uses an explicit stack to iteratively traverse the BVH on the GPU, avoiding recursion and mapping well to compute shader execution. A sample debug rendering used during development to confirm proper BVH-creation is shown in Fig. 1.

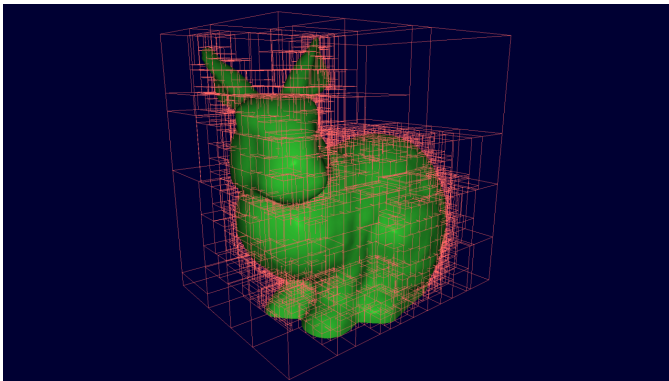


Figure 1: Debug render of the bunny model used to validate BVH-creation.

GGX microfacet BRDF Specular reflection is modeled using a microfacet-based BRDF with a GGX normal distribution function [7]. The microfacet model assumes that a surface consists of many small planar facets [5], therefore providing a physically plausible representation of surface roughness and produces long-tailed specular highlights [7].

The GGX normal distribution function is given by [7]:

$$D_{\text{GGX}}(h) = \frac{\alpha^2}{\pi [(n \cdot h)^2(\alpha^2 - 1) + 1]^2} \quad (3)$$

where h is the half-vector between incoming and outgoing directions, n is the surface normal, and α denotes surface roughness.

Shadowing and masking effects are accounted for using the Smith geometry function [2], and Fresnel reflectance is commonly approximated using Schlick's formulation [5]. Combined with importance sampling, this results in an energy-conserving and efficient specular reflection model [5, 2].

Visible Normal Distribution Function (VNDF) For microfacet BRDFs, the GGX normal distribution function D_{GGX} describes how microfacet normals are distributed on the surface and therefore defines the *shape of the specular lobe* [7]. However, when estimating the BRDF integral, sampling microfacet normals directly from D_{GGX} can be inefficient because many sampled microfacets are not visible from the view direction due to masking and shadowing [2].

VNDF sampling addresses this by sampling only *visible* microfacet normals conditioned on the view direction [3]. In this renderer, **GGX** is used to *evaluate* the physically based specular BRDF (i.e., it defines the material response), while **VNDF** is used to *sample* specular directions efficiently (i.e., it defines how random samples are drawn for Monte Carlo integration) [3, 1]. This reduces variance, especially for rough materials and grazing angles.

Multiple Importance Sampling (MIS). Monte Carlo estimators can sample light transport using different strategies, such as sampling according to the BRDF or sampling light sources directly [6, 5]. Each strategy performs well in certain cases but poorly in others [6]. For instance, BRDF sampling efficiently handles glossy reflections, while direct light sampling is more effective for small or distant light sources [6, 5].

Multiple importance sampling combines multiple sampling strategies into a single unbiased estimator by weighting each sample according to its probability of being generated by all strategies [6]. In this renderer, MIS is used to combine BRDF sampling with next event estimation for direct lighting [6, 5].

The power heuristic is used to compute the MIS weights [5]. For a sample generated by strategy i with probability density p_i , the MIS weight is given by:

$$w_i = \frac{(n_i p_i(x))^\beta}{\sum_j (n_j p_j(x))^\beta} \quad (4)$$

Using the power heuristic (with exponent $\beta = 2$) places more emphasis on the sampling strategy that better matches the integrand, which has been shown to reduce variance in many practical rendering scenarios, especially for glossy materials and high-frequency lighting [5].

Method

Pipeline overview The renderer is implemented as a progressive path tracer entirely inside an OpenGL compute shader. Each invocation traces one or more samples per pixel by generating primary rays from the camera and iteratively simulating their interaction with the scene [5]. Radiance contributions are accumulated across frames to achieve convergence [5].

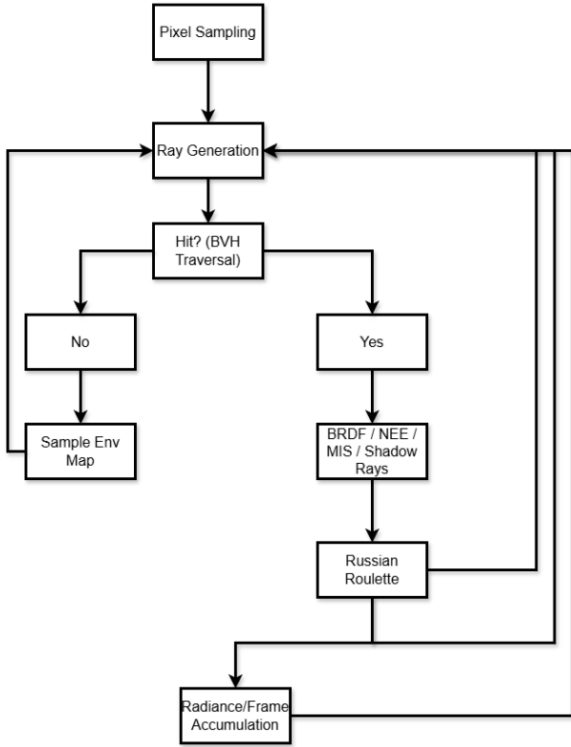


Figure 2: Compute-shader path tracing pipeline used in this project. Each iteration generates rays, traverses the BVH, evaluates BRDF and direct lighting (NEE/MIS), applies Russian roulette termination, and accumulates radiance over frames.

Pixel sampling and ray generation Using the sampled pixel position (ray origin and direction in world space), a primary ray is created from the camera. The *Pixel Sampling* and *Ray Generation* blocks in Fig. 2 correspond to this step.

Hit test via BVH traversal A bounding volume hierarchy is used to intersect the primary ray with the scene geometry. The BVH is uploaded to GPU buffers and constructed on the CPU (binning-based splitting). The compute shader uses an explicit stack (fixed maximum depth) to implement traversal iteratively. Before evaluating costly triangle intersections, cheap ray-AABB tests are used to remove a significant portion of the scene. This corresponds to Fig. 2’s *Hit? (BVH Traversal)* block.

Miss shading (environment) If the ray misses the scene, the renderer evaluates an environment term (sky / environment lighting) and terminates the path. The resulting radiance is added to the pixel estimate (*Sample Env Map* in Fig. 2).

Surface shading and direct lighting (NEE) Material parameters (metallic-roughness workflow with bindless textures) are retrieved if the ray strikes a surface. To estimate the outgoing radiance, combine: (i) indirect lighting by sampling the BSDF to spawn the next path segment, and (ii) direct lighting using Next Event Estimation (NEE) from a directional sun light with a shadow ray visibility test. Using the power heuristic, Multiple Importance Sampling (MIS) is combined with direct lighting and BSDF sampling [6]. This is equivalent to *BRDF / NEE / MIS / Shadow Rays* in Fig. 2.

Russian roulette termination To reduce needless work for low-contribution paths and maintain the estimator’s unbiasedness, paths are terminated using Russian roulette after a small minimum bounce count [5]. The inverse survival probability is used to scale the throughput of surviving paths. In Fig. 2, this is the *Russian Roulette* step.

Radiance and frame accumulation A limited number of samples per pixel are contributed by each frame. An incremental average is used to accumulate the result into a history buffer, allowing for progressive refinement while the application remains interactive (*Radiance/Frame Accumulation* in Fig. 2). An ACES filmic tonemapping curve is then used to map the accumulated image to displayable colors [4].

Results

Test scenes Complex triangle-mesh scenes with a range of material qualities (glossy, metallic, diffuse) were used to test the renderer. Due to occluded indoor geometry, the Bistro scene (Fig. 3) depicts a high-complexity environment with over 2.8 million triangles and numerous indirect light interactions. A popular benchmark scene with modest complexity (about 100k triangles) and big open areas that emphasize rough specular reflections and soft indirect illumination is the Sponza Atrium (Fig. 4).

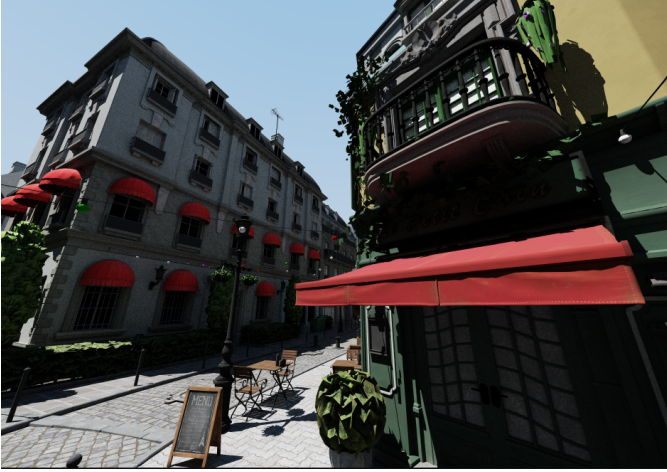


Figure 3: Rendered Bistro scene after 128 samples per pixel (approximately 2.8 million triangles).

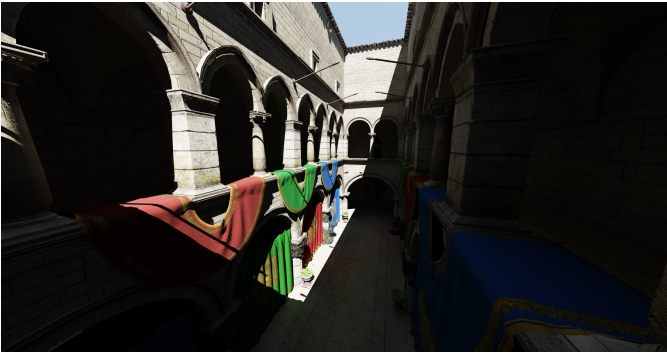


Figure 4: Rendered Sponza Atrium scene after 128 samples per pixel (approximately 100k triangles).

Performance evaluation CPU timers were used to measure execution time. A desktop computer with an AMD Ryzen 9 7900 CPU and an NVIDIA RTX 4090 GPU was used for performance tests. A set number of samples per frame and a maximum path depth that matched the sceneries in Fig. 3 and Fig. 4 were used to assess the renderer in interactive preview mode.

Note: Although evaluated on an RTX 4090, the implementation uses only OpenGL compute shaders (no RTX/RT-core API).

Table 1: Measured performance of the path tracer for different scenes on the test system.

Scene	Frame Time	Frame Rate
Bistro	≈ 28.57 ms	≈ 35 FPS
Sponza	≈ 13.69 ms	≈ 73 FPS

Conclusion

This project demonstrated how to use OpenGL compute shaders to construct a physically based path tracer in a totally shader-based manner. Global illumination effects were effectively calculated using Monte Carlo integration, BVH acceleration, and physically based shading models, even in the absence of specialized ray tracing hardware.

Performance limitations, noise reduction, and effective GPU-side traversal were the primary obstacles. While importance sampling, VNDF-based specular sampling, and multiple importance sampling greatly enhanced convergence, BVH acceleration was crucial for managing complicated scenes. Interactive previews were made possible while preserving an unbiased estimator by progressive frame accumulation.

By dividing traversal, shading, and sampling into distinct phases, **Wavefront Path Tracing** may enhance GPU occupancy from an architectural standpoint. The entire BVH building process might be shifted to the GPU. Realism might be further enhanced by more sophisticated light transport characteristics like depth of field, motion blur, subsurface scattering, and atmospheric rendering (a better sky-model).

Overall, this project demonstrates that without hardware ray tracing capabilities, a well-crafted shader-based path tracer can produce physically realistic results.

References

- [1] Mathieu Auziaffe. Vndf importance sampling for isotropic ggx distributions, 2024.
- [2] Eric Heitz. Understanding the masking-shadowing function in microfacet-based brdfs. *Journal of Computer Graphics Techniques*, 3(2), 2014.
- [3] Eric Heitz. Sampling the ggx distribution of visible normals. *Journal of Computer Graphics Techniques*, 7(4), 2018.
- [4] Krzysztof Narkowicz. Aces filmic tone mapping curve, 2015.
- [5] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann, 4 edition, 2024.

- [6] Eric Veach. *Robust Monte Carlo Methods for Light Transport Simulation*. PhD thesis, Stanford University, 1995.
- [7] Bruce Walter, Stephen R. Marschner, Hongsong Li, and Kenneth Torrance. Microfacet models for refraction through rough surfaces. *Proceedings of the Eurographics Symposium on Rendering*, 2007.